

PRESIDENCY UNIVERSITY

CSE321: Microprocessors Based System Design

Lecture 03: 8086 Internal Architecture

BIU, EU, Register Organization and Segmented Memory

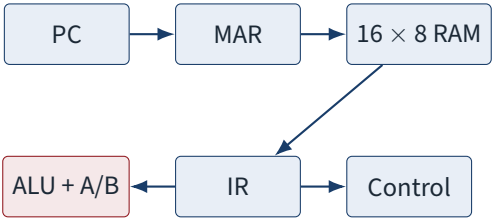
Afzal Hossan

Lecturer

Department of Computer Science and Engineering

Presidency University

SAP-1 gives us the basic mental model



- SAP-1 illustrates **program sequencing**, **stored instructions**, **buses**, and **ALU operations**.
- Its simple model fetches instructions and then executes them with very few registers.
- A practical processor needs more data handling, addressing capacity, flexible instructions, and quicker instruction supply.

Both machines fetch, decode and execute; the 8086 can overlap fetching with execution.

8086: three widths, three different meanings

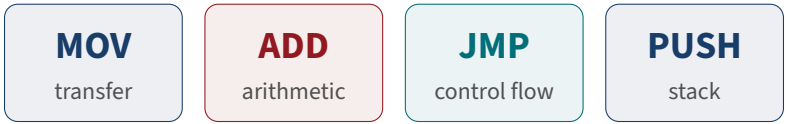
16-bit	**16-bit**	**20-bit**
ALU + primary registers	external data bus	physical address bus

- **16-bit processor:** AX, BX, CX, DX and ALU operations use 16-bit data; four data registers have addressable byte halves.
- **16-bit data bus:** transfers can involve one or two bytes, depending on alignment and access.
- **20-bit addresses:** can identify $2^{20} = 1{,}048{,}576$ **byte** locations $= 1$ MiB (conventionally called 1 MB).
- Intel introduced the 8086 in **1978**; it is part of the x86 architectural lineage.

Distinguish a register's *width*, a transfer's *width*, and a memory address's *width*: they answer three different questions.

From a small ISA to a richer instruction vocabulary

CISC | COMPLEX INSTRUCTION SET COMPUTER



What makes the 8086 CISC?

- Many instruction types and operand forms.
- Instructions occupy **variable numbers of bytes**.
- Registers, constants, and memory can be operands.

What changes inside the CPU?

- Decoder interprets unequal-size instruction encodings.
- The BIU supplies a **stream of bytes** for the EU.
- Some instructions use implicit registers.

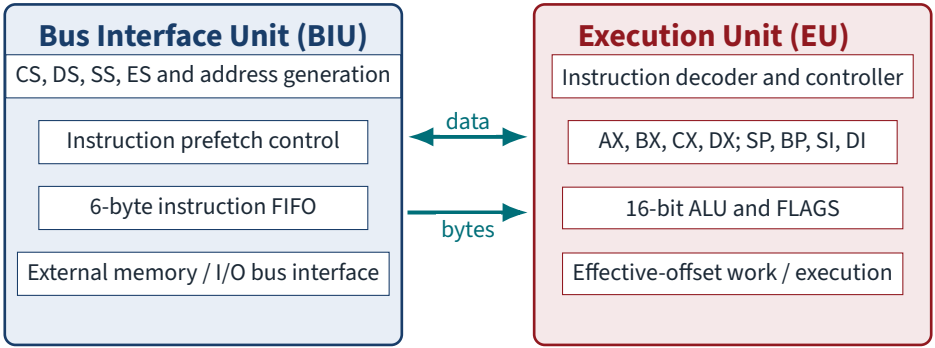
Examples: `MOV AX,BX` `ADD AX,5` `MOV AL,[SI]` `JMP label`

What changes from SAP-1 to 8086?

Property	SAP-1: teaching model	Intel 8086
Data path	8-bit A/B and bus	16-bit ALU and registers; 8-bit halves
Addressing	4-bit address; 16 byte locations	20-bit physical addresses; 1 MiB
Instructions	Few simple 8-bit forms	Rich variable-length CISC ISA
Registers	PC, MAR, IR, A/B, etc.	Data, pointer/index, segments, IP, FLAGS
Fetch and execute	Simple sequential model	BIU and EU can overlap activity
Instruction supply	No six-byte FIFO	Six-byte prefetch queue

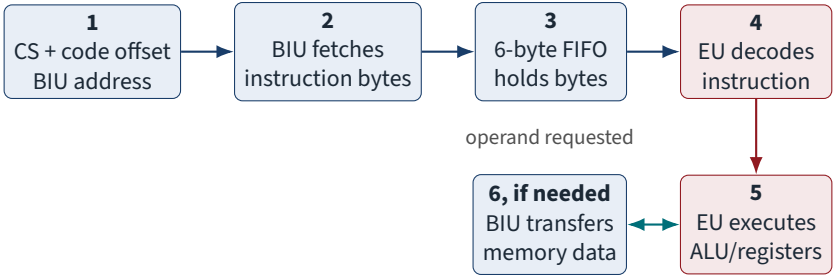
The analogy is functional, not exact: SAP-1's PC resembles CS:IP instruction sequencing; SAP-1's MAR does not map to one programmer-visible 8086 register.

Two units, one processor: BIU + EU



The BIU speaks to memory / I/O. The EU decodes and executes instructions.

Follow an instruction from memory to the ALU



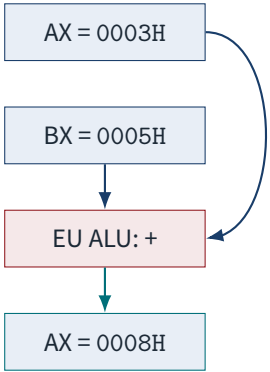
- All external memory and I/O transfers are serviced by the BIU, not directly by the EU.
- Arithmetic may be fast while instruction bytes or operands still require bus activity.

Inside the EU: decode and execute

Register-only example

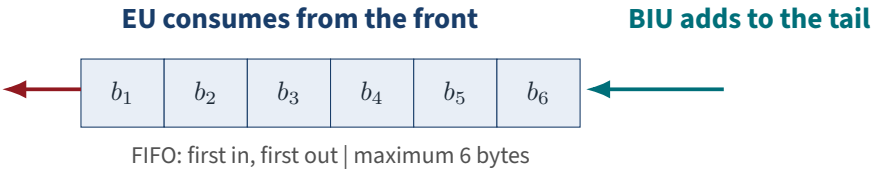
```
MOV AX,0003H
MOV BX,0005H
ADD AX,BX
```

The EU decodes the operations and adds $3 + 5 = 8$ in its ALU. AX becomes 0008H.



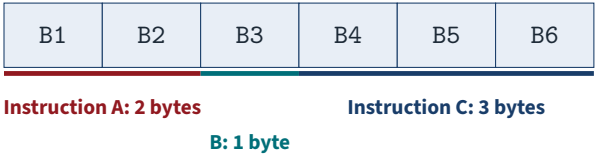
Register arithmetic uses EU; memory operands also require BIU access to external memory.

The six-byte instruction queue



- The BIU fetches **instruction bytes in advance** when the bus is available.
- The EU removes instruction bytes in order as it decodes them.
- The queue stores **up to 6 bytes**, not necessarily 6 instructions.
- The queue is a prefetch buffer, **not** a general-purpose data cache.

The queue stores bytes, not instructions

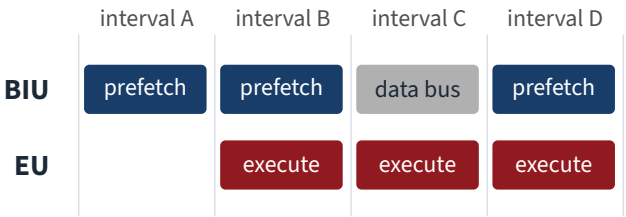


Reasoning

A hypothetical stream of instructions of sizes **2**, **1**, and **3** bytes fills the six queue positions with **three** instructions. Actual 8086 instruction lengths depend on operation, operands and prefixes.

The diagram is an instructional grouping of generic bytes, not a claim about particular encoded opcodes.

Prefetch and execution can overlap

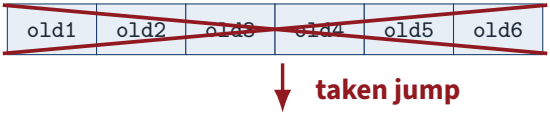


- While the EU executes a register-based instruction, BIU may refill the queue.
- When EU needs memory data, the shared external bus may serve that demand instead of prefetch.
- This is **fetch/execute overlap**, not a fixed one-instruction-per-clock pipeline.

Conceptual overlap only: this is not a clock-accurate bus timing diagram.

Taken branch? Discard the old bytes

Before taken branch:



After target change:

new1	new2	new3	new4	new5	new6

The cause

The queued bytes belong to the *old sequential path*. A taken control transfer changes the next CS:IP, so the queue is invalidated and the BIU fetches bytes at the target. The BIU prefetch position may otherwise be ahead of the EU's instruction position.

Code fetch and data access use different addresses

Assume $CS = 2200\text{H}$, $IP = 0040\text{H}$, $DS = 3300\text{H}$, and a data operand at offset 0120H.

1. **Instruction side:** BIU generates code address $22000\text{H} + 0040\text{H} = \boxed{22040\text{H}}$, then fetches code bytes.
2. **Decode:** EU consumes queue bytes and recognizes an instruction that requires memory data.
3. **Operand side:** BIU generates data address $33000\text{H} + 0120\text{H} = \boxed{33120\text{H}}$.
4. **Execute:** BIU brings operand data; EU performs the requested operation and updates the appropriate flags.
5. **Continue:** EU resumes instruction consumption; BIU prefetches whenever the external bus permits.

Two addresses, one CPU: CS:IP selects code; DS:offset selects ordinary data. Detailed operand addressing modes come in Lecture 06.

Register file: the full 8086 map

Data registers (4)	Pointer / index (4)	Segments (4)
AX accumulator	**SP** stack ptr.	**CS** code
BX base	**BP** base ptr.	**DS** data
CX counter	**SI** source idx.	**SS** stack
DX data	**DI** dest. idx.	**ES** extra

IP = instruction pointer

FLAGS = status + control bits

Count correctly

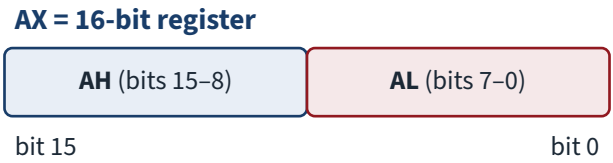
Four data + four pointer/index + four segment + one IP + one FLAGS $= \boxed{14}$ distinct 16-bit registers in this overview. The eight names AH/AL/BH/BL/CH/CL/DH/DL refer to *halves*, not eight additional independent registers.

AX, BX, CX, DX: the data register family

Register	8-bit halves	Typical use (not an exclusive restriction)
AX	AH, AL	Accumulator in arithmetic; implied operand for many operations and some I/O.
BX	BH, BL	General data; “base” register for certain memory forms.
CX	CH, CL	Loop count and repetition; CL is used for variable shift counts on original 8086.
DX	DH, DL	Extension for multiplication/division; DX in variable I/O-port addressing.

All four can hold ordinary 16-bit values. Their names hint at *common hardware/instruction conventions*; they do not prohibit general computation.

One 16-bit register, two accessible bytes



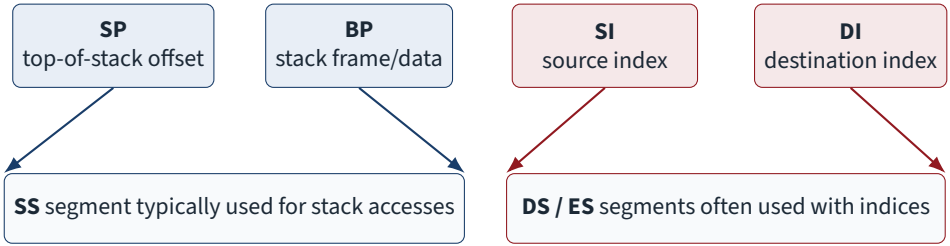
Example: AX = 4B2CH ⇒ AH = 4BH, AL = 2CH

A write to AL changes only half of AX

If AX = `4B2CH` and `MOV AL,91H` executes, AX becomes `4B91H`. AH remains `4BH`.

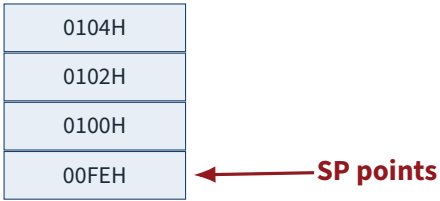
BX has BH/BL; CX has CH/CL; DX has DH/DL. SP, BP, SI, DI and segment registers do not have addressable high/low halves.

Pointer and index registers carry offsets



- An **offset** identifies a location *within a selected segment*.
- SP moves with stack pushes/pops; BP commonly refers to stack-frame operands.
- SI and DI aid indexed and string operations. Defaults and exceptions are explained after segmentation.

SP and BP are not interchangeable



- Stack uses segment **SS** and an offset, commonly **SP**.
- On a 16-bit `PUSH`, SP normally decreases by **2** before the word is stored.
- **BP** enables convenient access to data near the stack top; BP-based references default to SS.

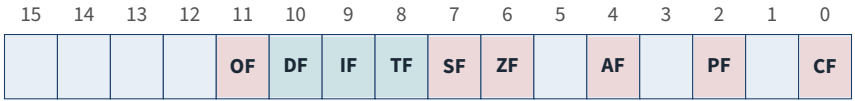
Why the stack grows downward is discussed only to explain the meaning of SP; CALL/RET programming is developed in Lecture 07.

Segments and IP: which pair accesses what?

Name	Role	Typical pairing
CS	Code segment base for instruction fetch.	CS:IP
DS	Default base for many ordinary data memory operands.	DS:BX, DS:SI
SS	Stack-segment base for stack accesses and BP-based references.	SS:SP, SS:BP
ES	Extra data segment; important for string destination.	ES:DI
IP	Offset of architecturally next instruction within CS.	CS:IP (not a standalone address)

The four segment registers contain *16-bit segment values*, not ready-made 20-bit physical bases. The hardware shifts the segment value left by four bits when making an address.

Inside FLAGS: named bit positions



STATUS FLAGS: 6

CF, PF, AF, ZF, SF, OF

Describe results of arithmetic / logic where defined.

CONTROL FLAGS: 3

TF, IF, DF

Influence processor behavior.

Bit positions 1, 3, 5, 12–15 are not assigned the nine flag meanings in the original 8086 description. Flag effects depend on the particular instruction.

Status flags: six ways to describe a result

Flag	Meaning	When set (simplified)
CF (0)	Carry flag	Carry out of unsigned addition / borrow from unsigned subtraction; shift/rotate semantics vary.
PF (2)	Parity flag	Least-significant result byte has an **even** number of one bits.
AF (4)	Auxiliary carry	Carry/borrow crossing bit 3 to bit 4; useful for BCD adjustment.
ZF (6)	Zero flag	Result of an operation is zero.
SF (7)	Sign flag	Copies the most significant bit of the result at its operand width.
OF (11)	Overflow flag	Signed arithmetic result cannot fit the operand's signed range.

These are conceptual rules for arithmetic examples. Not all operations update every flag, and some leave a flag unchanged or undefined. Instruction-specific rules are reserved for Lecture 05.

Control flags: change processor behavior

TF (bit 8)	IF (bit 9)	DF (bit 10)
Trap flag: when enabled, requests single-step trap behavior for debugging.	**Interrupt-enable flag**: governs maskable hardware interrupt recognition; not NMI.	**Direction flag**: affects how SI/DI move in string operations: 0 increment, 1 decrement.

An intuitive DF picture

DF = 0 address $\rightarrow$ higher address **DF = 1** address $\rightarrow$ lower address

Flags do not all serve the same purpose: six summarize result conditions, while three influence control of execution.

Carry and overflow measure different things

8-bit value interpreted UNSIGNED

range: 0 to 255

7FH means 127

FFH means 255

Same bits interpreted SIGNED

range: -128 to $+127$

7FH means $+127$

FFH means -1

CF answers an **unsigned** arithmetic question. OF answers a **signed** arithmetic question. The stored bits can be identical while the two interpretations differ.

One result, two number systems: CF versus OF

INTERPRET THE SAME 8-BIT RESULT AS UNSIGNED AND SIGNED

`7FH + 01H = 80H`
unsigned: $127 + 1 = 128$ fits
signed: $+127 + 1$ exceeds $+127$

CF = 0 OF = 1

`FFH + 01H = 00H`
unsigned: $255 + 1 = 256$ wraps
signed: $-1 + 1 = 0$ fits

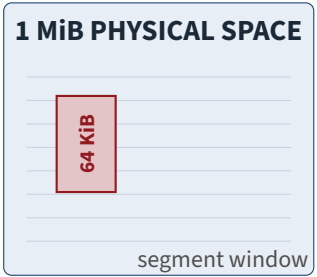
CF = 1 OF = 0

Rule: CF reports unsigned carry/borrow; OF reports signed-range overflow. Also: 80H has SF=1, ZF=0; 00H has SF=0, ZF=1, PF=1.

Why 16-bit offsets are not the whole address

The constraint

- IP and many offset registers are **16 bits**.
- A lone 16-bit offset spans $2^{16} = 65{,}536$ bytes (64 KiB).
- But the 8086 has **20 address lines**, allowing 1 MiB of byte addresses.



Solution: use a **16-bit segment value** to select a base, then a **16-bit offset** within that window. The BIU combines them into the physical address.

How the 8086 constructs a physical address

THE MOST IMPORTANT ADDRESSING RELATIONSHIP



$$PA = ((S \ll 4) + O) \bmod 2^{20}$$

S and O are both 16-bit values; $S \ll 4$ is the segment base (16-byte aligned).

The 20-bit external address spans `00000H`–`FFFFFH` (1 MiB). Retain the low 20 bits if addition crosses `FFFFFH`.

Four segment registers, four common roles



Each memory access combines its chosen segment register with a relevant offset.

- **CS**: code fetching; **DS**: ordinary data; **SS**: stack; **ES**: extra data, especially string destinations.
- These are **roles and defaults**, not four separate physical RAM chips.

Compute a physical address: 1234:5678

Given logical address: $1234:5678$

1. Segment = 1234H, offset = 5678H.
2. Convert segment to base: $1234\text{H} \times 16 = 12340\text{H}$.
3. Add offset: $12340\text{H} + 05678\text{H} = \boxed{179B8\text{H}}$.
4. Sum is below 100000H; no wrap is needed.

Algorithm worth memorizing

Write the *segment with one zero hex digit appended*, align the offset underneath, perform hexadecimal addition, and retain the low 20 bits.

Code and data can live in different segments

Assume $CS = 2000\text{H}$, $IP = 0100\text{H}$, $DS = 2800\text{H}$, $BX = 0040\text{H}$.

Instruction fetch: CS:IP

$$20000\text{H} + 00100\text{H}$$
$$= \boxed{20100\text{H}}$$

Ordinary data: DS:BX

$$28000\text{H} + 00040\text{H}$$
$$= \boxed{28040\text{H}}$$

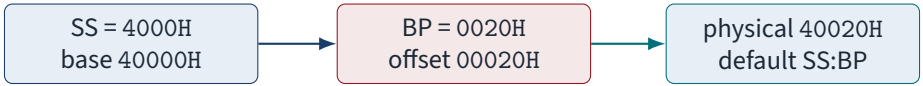
The offset name alone does not tell the whole address. A **segment selection rule** is required. The same numeric offset can point to different physical locations with different segment values.

Default segment rules you must know

Fetch and ordinary data	Stack and string operations
Instruction fetch: CS:IP	**Stack PUSH/POP: SS:SP**
Ordinary [BX], [SI], [DI]: DS	**String source: DS:SI**
Data forms involving BP: SS	**String destination: ES:DI**

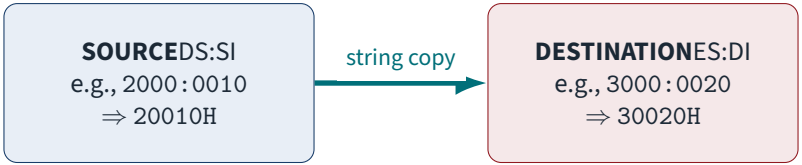
Default segments are conventions of the addressing form or instruction. Segment overrides exist for many data references, while certain string destinations remain ES on the original 8086. Detailed encoding rules come in Lecture 06.

The BP exception: default segment is SS



- When a memory addressing form uses BP, the default segment is **SS**, not DS.
- SP supports implicit stack operations; BP supports accessing stack-frame values and can appear in explicit addressing forms.
- A register value such as 0020H is **not** a complete physical address.

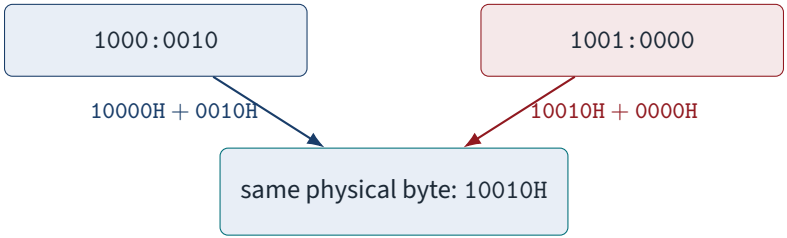
Why string destinations use ES:DI



- A string/memory-block operation may read a source byte via **DS:SI** and store it via **ES:DI**.
- SI and DI change according to the direction flag (DF) and data width of the string operation.

This is an architectural motivation for ES, not a complete string-instruction programming lesson.

Two logical addresses can name the same byte



This is called **aliasing**: several segment:offset pairs may refer to the same physical location. Segments are not necessarily disjoint or protected memory regions.

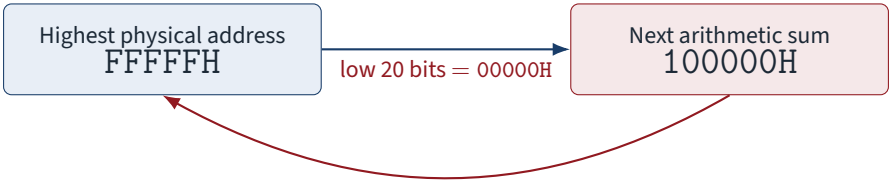
Segment windows overlap in physical memory



- A segment selects a **starting base**; offsets select up to 64 KiB relative to it.
- Since segment bases move in **16-byte increments**, windows often overlap strongly.

Illustration is not to scale. Neither segment is an isolated allocation in hardware.

20-bit arithmetic wraps at 1 MiB



Example on original 8086

$$\texttt{FFFF:0010} \Rightarrow \texttt{FFFF0H} + \texttt{0010H} = \texttt{100000H}$$

The physical address is $\boxed{\texttt{00000H}}$ after 20-bit wraparound.

The arithmetic sum can use 21 bits, but the original 8086 exposes only 20 physical address bits.

A PUSH changes SP before storing data

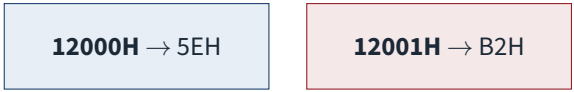
Initial state: SS = 4000H, SP = 0100H. Perform a 16-bit PUSH.

1. 8086 stack grows toward lower offsets: SP $\leftarrow$ SP -2.
2. New SP = 0100H $-$ 0002H $=$ $\boxed{\texttt{00FEH}}$.
3. The word is written beginning at SS:SP = 4000 : 00FE.
4. Physical address = 40000H $+$ 00FEH $=$ $\boxed{\texttt{400FEH}}$.

A word has two bytes; the low byte is placed at the lower physical address on 8086. The companion high byte follows it. Detailed stack programming appears in Lecture 07.

Byte addresses and little-endian words

Store word B25EH starting at physical address 12000H:

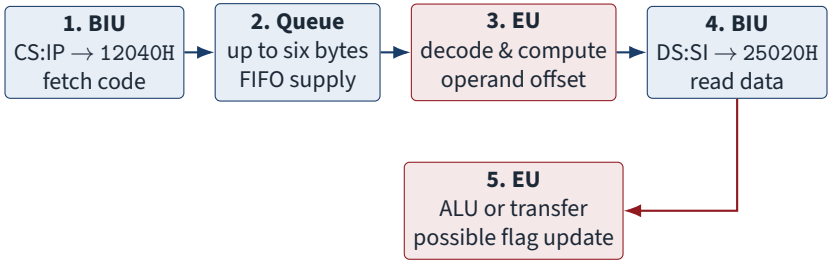


lower address = low byte | higher address = high byte

- The 8086 uses **byte-addressed** memory: consecutive addresses refer to consecutive bytes.
- **Little-endian** word storage puts low-order byte first in increasing address order.
- Address calculation locates the first byte of an operand; separate byte locations hold larger data.

Trace one instruction across the full system

Given $CS = 1200\text{H}$, $IP = 0040\text{H}$, $DS = 2500\text{H}$, $SI = 0020\text{H}$.



This is a conceptual trace for an instruction requiring a data source at DS:SI. Real instruction encodings, timing and flag effects depend on the selected operation.

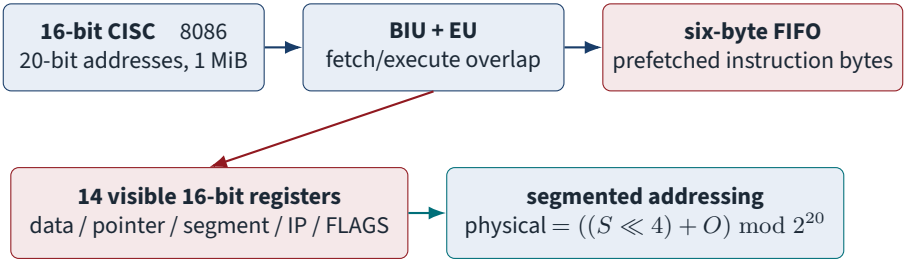
Try these four 8086 challenges

1. Classify AX, BP, ES, IP and FLAGS by register category.
2. If AX = `A73CH` and AL is changed to `42H`, what is AX?
3. Find the physical address for `2500:03A0`.
4. Do `1000:0010` and `1001:0000` identify the same byte? Show the sum.
5. What happens to queued sequential bytes after a taken branch?

Check your reasoning

1. AX: general data; BP: pointer; ES: segment; IP: instruction pointer; FLAGS: status/control word.
2. AH remains A7H, so AX becomes $\boxed{\texttt{A742H}}$.
3. $25000\text{H} + 03\text{A0H} = \boxed{253\text{A0H}}$.
4. Yes: $10000\text{H} + 0010\text{H} = 10010\text{H}$; $10010\text{H} + 0000\text{H} = 10010\text{H}$.
5. The prefetch queue is discarded/refilled for the branch target; old-path sequential bytes are not the next code stream.

The architecture in one picture



Think of an instruction as a *stream of bytes*, an EU computation as a *register/ALU action*, and memory access as a *segment + offset translated by the BIU*.